Cost-Aware Dual-Quantization KV Execution in HBM-PIM

Idea Summary:
Long-context LLM serving increasingly relies on quantized KV caches to reduce memory bandwidth pressure. Existing methods do one of two things: 1) quantize the KV cache in memory, send it to the processor (GPU or other accelerator), and have the processor dequantize before using that KV cache information. 2) they always operate directly on a quantized representation of the KV cache.

My idea is to have an HBM-PIM architecture where decisions are made regarding each KV cache block. The PIM can choose to either operate on a quantized version of the block or dequantize the block and operate on it with hardware supporting higher-precision operations. We could have a controller in the HBM base die that can decide when to select one of these options over the other. The key thing is to evaluate when the cost of dequantization is worth it compared to computing on the compressed representation.

The workload that this can be useful for is sparse attention since it doesn't use all elements in the KV cache for all decode tokens.

- Think of what other workloads will benefit from this idea

Side notes:

- We would want to dequantize when there is high confidence that the KV cache block will be used very often–amortize the cost of dequantization over many uses
    - Not too sure how to find that
- I asked GPT deep research to see if there are any papers that already do this out there, and it said it couldn't find anything…

What I need to do:

1. Find some kind of simulator that supports PIM (GPT tells me that Ramulator2 and AttAcc are good choices)
    a. Need to compare between always compressed, always dequantize, static ratio of dequantization vs compression, and dynamic dequantization policies across a variety of long-context and long-reasoning workloads.
    b. What to measure: latency, throughput, energy

Related Papers:

1. TokenStack: A Heterogeneous HBM-PIM Architecture and Runtime for Efficient LLM Inference (2026)

Token stack proposes a heterogeneous HBM organization with PIM enabled-layers for hot KV blocks and dense-capacity layers for colder KV blocks. They use a base-die controller to move

the KV blocks around, place them in certain regions of memory, and do in-memory quantization. The difference compared to my proposal is that the quantization is used for storage and data movement rather than dynamically choosing between executing on the compressed data or dequantizing it before execution.

2. AQPIM: Breaking the PIM Capacity Wall for LLMs with In-Memory Activation Quantization (HPCA 2026)

AQPIM uses product quantization to compress KV-cache activations inside PIM and does attention computations directly on the compressed representation of the data. Similar to my idea but it does not include a dynamic selection between computing on the quantized data or dequantizing before computation.

3. QEvict: Recoverable Quantized KV Eviction for Attention-Drift-Robust Long-Context Decoding (2026)

QEvict maintains full-precision, quantized-recoverable, and evicted KV tiers. It dynamically dequantizes and promotes a quantized block when its attention importance increases during decode. This is similar to my idea since it dynamically selects between computing on compressed KV and computing on dequantized KV, but it differs in the sense that the selection decision is based on attention and it does not use HBM-PIM.

4. P³-LLM: An Integrated NPU-PIM Accelerator for Edge LLM Inference Using Hybrid Numerical Formats (ISCA 2026)

P³-LLM co-designs a PIM accelerator and mixed-precision quantization so that low-precision LLM operands, including KV-cache data, can be processed with specialized PIM hardware while minimizing the dequantization overhead during decode. While it does do mixed-precision execution, it differs from my idea in the sense that its numerical precision formats and execution paths are predetermined instead of being dynamically selected per KV block.